Hybrid Cloud Monitoring & Alerting System

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

Abhiram Majeti-2320030108

Swaran chandra-2320030337

Sai Aditya Rao-2320090013

Under the Esteemed Guidance of

PRATHUSHA BONAGANI

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Declaration

The Project Report entitled "Cloud-Based Financial Transaction Pattern Analysis" is a record of Bonafide work of team members Abhiram-2320030108, Swaran chandra-2320030337, and Sai Aditya Rao-2320090013, submitted in partial fulfilment for the award of B. Tech in Computer Engineering to K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Abhiram Majeti-2320030108
Swaran chandra-2320030337
Sai Aditya Rao-2320090013

K L (Deemed to be) University DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Certificate

This is to certify that the project-based report entitled "Hybrid Cloud Monitoring & Alerting System" is a Bonafide work carried out and submitted by Abhiram Majeti-2320030108, Swaran chandra-2320030337, and Sai Aditya Rao-2320090013, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science Engineering, at K L (Deemed to be University), during the academic year 2025–2026.

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, **Prathusha Bonagani**, Department of Computer Science which cannot be expressed in words for his tremendous support, encouragement and guidance for this project.

We express our gratitude to Dr. P Venkateswara Rao, Head of the Department for Computer Science Engineering for providing us with adequate facilities, ways and means by which we can complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

Abhiram Majeti-2320030108
Swaran chandra-2320030337
Sai Aditya Rao-2320090013

TABLE OF CONTENTS

S.No	Contents	Page no
1	Abstract	6
2	Introduction	7
3	Literature survey	9
4	Hardware and Software requirements	11
5	Implementation	12
6	Experimentation and Code	16
7	Results	23
8	Conclusion	28
19	References	31

Abstract

Cloud computing has become the backbone of modern IT infrastructure, enabling organizations to deploy scalable and flexible systems. With the increasing reliance on cloud-hosted virtual machines, continuous monitoring has become essential to ensure system performance, availability, and security.

This project presents a Hybrid Cloud Monitoring and Alerting System that integrates Microsoft Azure's native monitoring capabilities with custom-built Python-based monitoring scripts. The system is designed to monitor key system parameters such as CPU utilization, memory usage, disk space availability, and unauthorized login attempts.

Azure Monitor is used to track infrastructure-level metrics and generate alerts, while Python scripts provide deeper insight into operating system-level activities such as authentication logs and resource consumption. The monitoring system is automated using cron jobs, ensuring periodic execution without manual intervention.

The project also demonstrates real-time alert generation through controlled experimentation, such as CPU stress testing and simulated login failures. The results show that the hybrid approach significantly enhances monitoring accuracy, improves security awareness, and enables proactive system management.

This project highlights the importance of combining cloud-native tools with custom monitoring solutions to achieve a comprehensive and efficient monitoring system suitable for real-world enterprise environments.

Introduction

2.1 Background

Cloud computing has revolutionized the way applications and services are deployed. Instead of relying on physical servers, organizations now use virtual machines hosted on cloud platforms such as Microsoft Azure. These virtual machines provide scalability, cost efficiency, and high availability. However, managing cloud infrastructure introduces new challenges, particularly in monitoring system performance and ensuring security. Without proper monitoring, systems may experience:

- Performance degradation
- Resource exhaustion
- Unauthorized access
- Service downtime

2.2 Need for Monitoring

Monitoring is essential for:

- Maintaining system health
- Detecting anomalies
- Preventing failures
- Ensuring security compliance

Traditional monitoring systems focus mainly on high-level metrics such as CPU usage and memory consumption. However, deeper system-level monitoring is required to detect issues such as:

- Failed login attempts
- Unauthorized access
- Disk usage anomalies
- Resource misuse

2.3 Problem Statement

Azure Monitor provides powerful infrastructure monitoring capabilities, but it has limitations:

- Limited access to OS-level logs
- No direct monitoring of authentication events

- Lack of custom monitoring logic
- Limited flexibility for automation

2.4 Proposed Solution

This project proposes a **Hybrid Monitoring System** that combines:

- Azure Monitor → Infrastructure-level monitoring
- Python Scripts → OS-level monitoring

This approach ensures:

- Complete system visibility
- Improved security monitoring
- Flexible alerting mechanisms

2.5 Objectives

The main objectives of the project are:

- To monitor system performance (CPU, memory, disk)
- To detect security threats (login failures)

Literature survey

Cloud monitoring has been widely studied in both academic research and industry applications. Several tools and frameworks have been developed to address monitoring challenges.

3.1 Cloud Monitoring Tools

Azure Monitor

Azure Monitor provides real-time metrics, logging, and alerting features. It is widely used in enterprise environments.

Advantages:

- Easy integration with Azure services
- Real-time alerting
- Scalable monitoring

Limitations:

- Limited OS-level visibility
- Requires additional configuration for logs

AWS CloudWatch

AWS CloudWatch provides monitoring for AWS resources.

Advantages:

- Scalable monitoring
- Integration with AWS ecosystem

Limitations:

- Complex setup
- Limited customization

Google Cloud Monitoring

Provides monitoring and visualization tools.

Advantages:

- Good dashboard support
- Integration with Google services

Limitations:

- Limited customization for OS-level monitoring

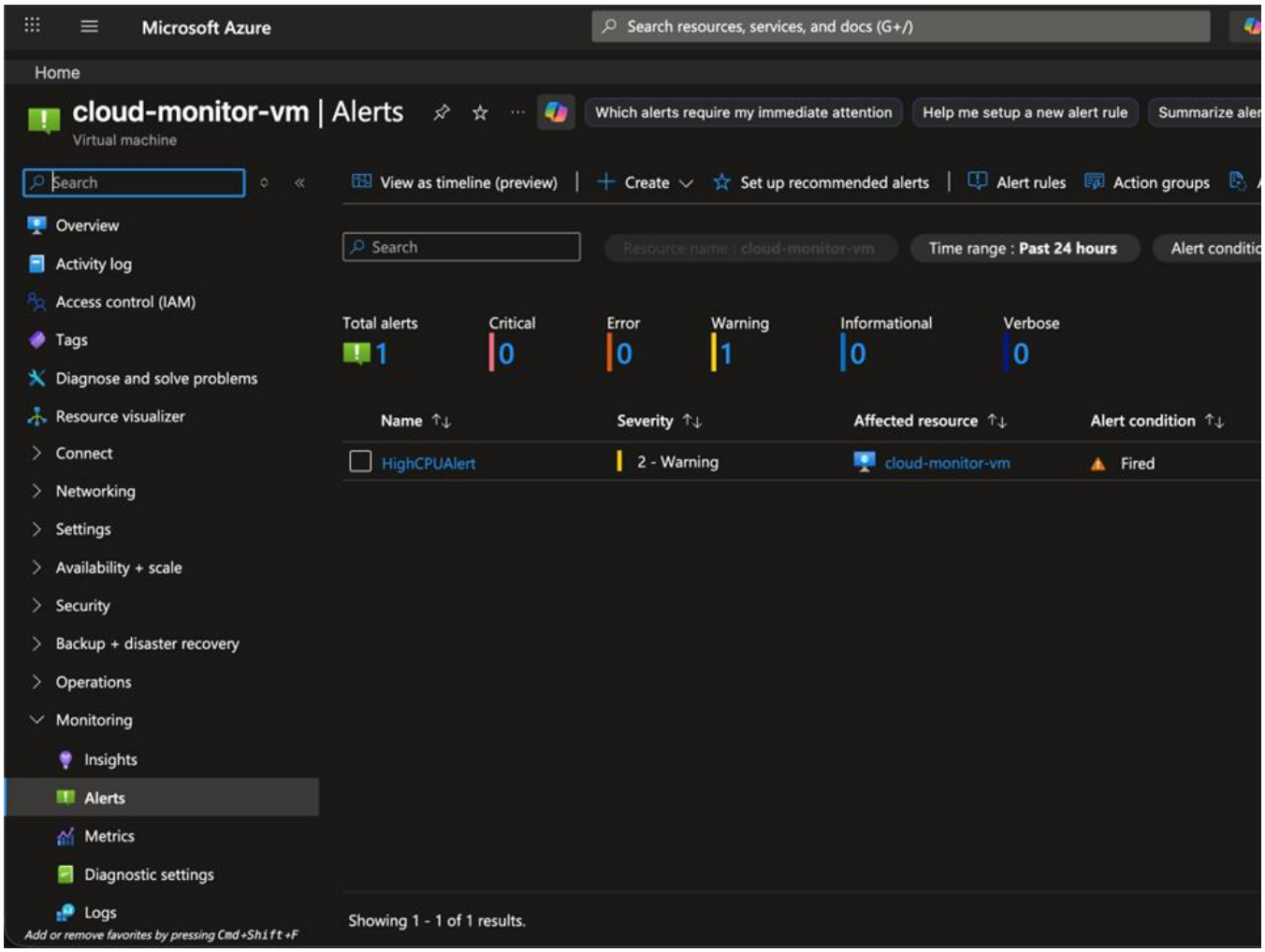
3.2 Research Observations

Studies show that:

- Cloud monitoring tools are effective for infrastructure
- Custom scripts are needed for detailed monitoring
- Hybrid systems provide better performance

3.3 Conclusion

The literature clearly indicates that combining cloud monitoring with custom scripting is the most effective approach.



Hardware and Software requirements

Hardware Requirements

- Processor: Intel i3 or above
- RAM: Minimum 4 GB (8 GB recommended)
- Storage: At least 500 MB free space
- Internet connectivity for cloud access

Software Requirements

- Operating System: Ubuntu (on Azure VM)
- Cloud Platform: Microsoft Azure
- Tools and Technologies:
  - Azure Monitor
  - Python (3.x)
  - Cron (Linux Scheduler)
- Libraries:
  - psutil (for system monitoring)
  - smtplib (for email alerts)

SYSTEM ARCHITECTURE

The system consists of the following components:

- **Azure Virtual Machine (Ubuntu)**
- **Azure Monitor (Infrastructure Monitoring)**
- **Python Scripts (OS Monitoring)**
- **Cron Jobs (Automation)**
- **Email Alert System**

Working Flow

1. Azure Monitor tracks VM-level metrics
2. Python scripts monitor OS-level parameters
3. Cron jobs execute scripts periodically
4. Threshold conditions are checked
5. Alerts are triggered and sent via email

This creates a **dual-layer monitoring system**.

IMPLEMENTATION

5.1 System Overview

The Hybrid Cloud Monitoring and Alerting System is implemented using a combination of **Microsoft Azure services** and **Python-based monitoring scripts**. The system follows a layered architecture where monitoring is performed at both infrastructure and operating system levels.

The implementation is divided into multiple components:

- Cloud Infrastructure Setup (Azure VM)
- Monitoring Script Development
- Automation using Cron Jobs
- Alert Integration using Azure Monitor
- Log Generation and Analysis

5.2 Cloud Infrastructure Setup

The system is deployed on a virtual machine created in Microsoft Azure.

Virtual Machine Configuration

- Operating System: Ubuntu Server 24.04 LTS
- Size: Standard B1s (or similar)
- Authentication: SSH Key-based login
- Public IP: Enabled

The VM acts as the environment where monitoring scripts are executed.

5.3 Environment Setup

To run monitoring scripts, a Python environment is configured.

Steps:

1. Install Python
2. Create virtual environment
3. Install required libraries

```
python3 -m venv monitor-env
source monitor-env/bin/activate
pip install psutil requests
```

This ensures isolated and efficient execution of scripts.

5.4 Monitoring Script Design

The system consists of multiple Python scripts, each designed for a specific monitoring task.

5.4.1 CPU Monitoring

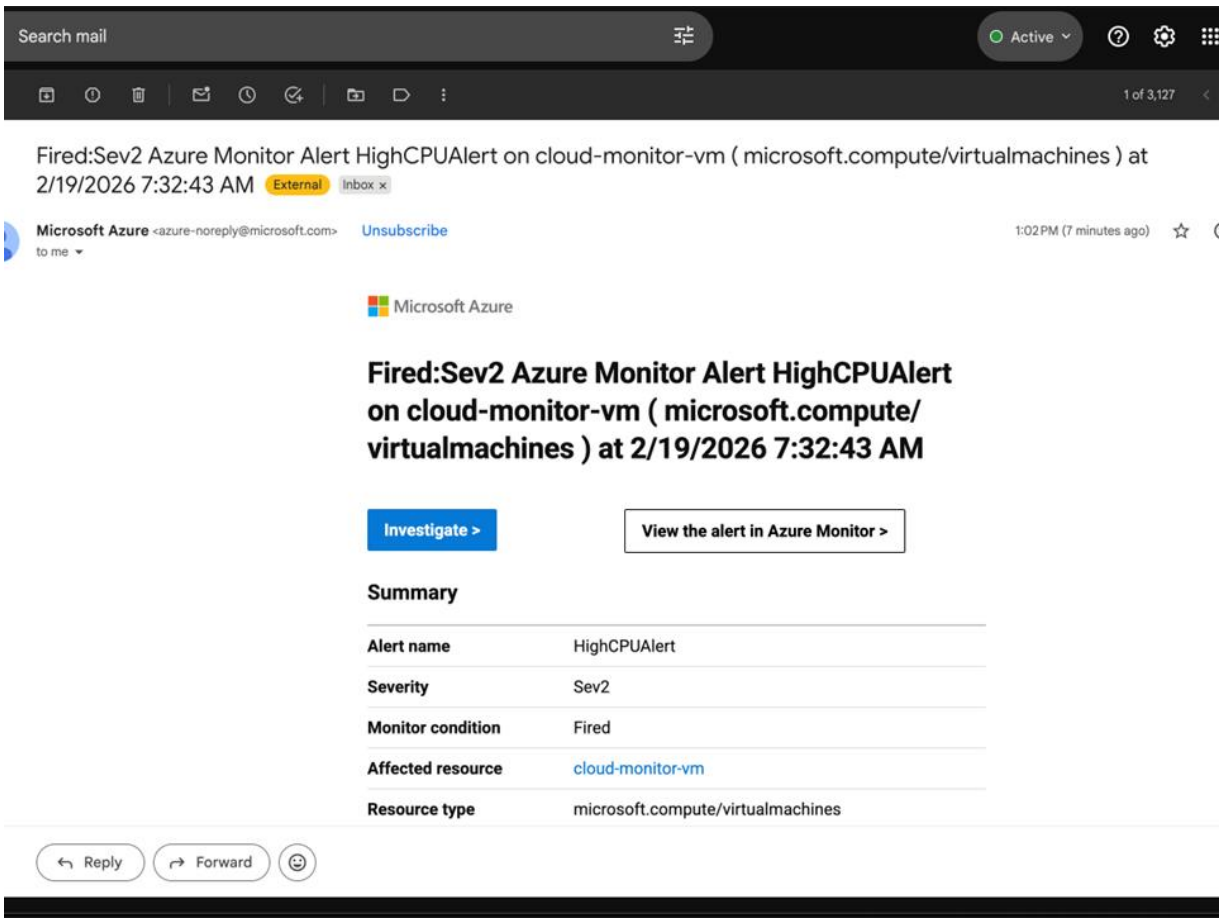
The CPU monitoring script continuously checks CPU utilization using the psutil library.

Working:

- Retrieves CPU usage percentage
- Compares with threshold (80%)
- Generates alert if exceeded

Logic:

```
cpu = psutil.cpu_percent(interval=1)
if cpu > 80:
    print("High CPU Alert")
```



5.4.2 Disk Monitoring

The disk monitoring script checks available disk space.

Working:

- Retrieves total and available disk space
- Calculates usage percentage
- Generates alert if disk space is low

5.4.3 Memory Monitoring

The memory monitoring script tracks RAM usage.

Working:

- Reads memory utilization
- Detects overload conditions
- Alerts if memory exceeds threshold

5.4.4 Login Monitoring

This is a security-focused script that detects failed login attempts.

Working:

- Reads system log file /var/log/auth.log
- Searches for failed login entries
- Generates alert if suspicious activity is found

5.5 Automation Using Cron

To ensure continuous monitoring, scripts are scheduled using cron jobs.

Configuration:

crontab -e

Example entries:

*/5 * * * * cpu_monitor.py

*/5 * * * * disk_monitor.py

*/10 * * * * login_monitor.py

Advantages:

- Eliminates manual execution
- Ensures periodic monitoring
- Improves system reliability

5.6 Azure Monitor Integration

Azure Monitor is configured to track CPU usage and generate alerts.

Steps:

1. Create alert rule
2. Set condition (CPU > 80%)
3. Configure action group (email notification)

Purpose:

- Detect high resource usage
- Notify administrator immediately

5.7 System Workflow

The system operates as follows:

1. Scripts run automatically via cron
2. System metrics are collected
3. Threshold conditions are evaluated
4. Alerts are generated if needed
5. Logs are stored for analysis

EXPERIMENTATION AND CODE

6.1 Objective of Experimentation

The objective of this phase is to validate the working of the Hybrid Cloud Monitoring System by performing real-time testing and observing how the system behaves under different conditions.

The experiments are designed to:

- Verify the accuracy of monitoring scripts
- Test system response under high load conditions
- Validate alert generation mechanism
- Evaluate automation using cron jobs
- Analyse system logs and outputs

6.2 Experimental Setup

The experiments were conducted on a virtual machine deployed on Microsoft Azure.

Environment Details:

- Platform: Microsoft Azure
- OS: Ubuntu Server 24.04
- Monitoring Language: Python 3
- Libraries Used: psutil, requests
- Automation Tool: Cron

The scripts were executed **both manually and through automated scheduling to observe system behaviour in real-time.**

6.3 CPU Monitoring Experiment

6.3.1 Purpose

To verify whether the system can detect high CPU usage and generate alerts when the threshold is exceeded.

6.3.2 Code Explanation

```
import psutil
```

```
CPU_THRESHOLD = 80

cpu = psutil.cpu_percent(interval=1)

if cpu > CPU_THRESHOLD:
    print(f"High CPU Usage Alert: {cpu}%")
else:
    print(f"CPU usage normal: {cpu}%")
```

Explanation:

psutil.cpu_percent() measures CPU utilization

The threshold is set to 80%

If CPU exceeds threshold, alert is generated

6.3.3 Experiment Procedure

To simulate high CPU usage, multiple background processes were created:

```
yes > /dev/null &
yes > /dev/null &
yes > /dev/null &
```

These commands continuously consume CPU resources.

6.3.4 Observations

- CPU usage increased to nearly 100%
- Script detected threshold breach
- Alert message printed
- Azure Monitor triggered alert

6.3.5 Result

The CPU monitoring system successfully detected high CPU usage and generated alerts in real time.

6.4 Disk Monitoring Experiment

6.4.1 Purpose

To verify whether the system detects low disk space conditions.

6.4.2 Code Explanation

```
import psutil

DISK_THRESHOLD = 20

disk = psutil.disk_usage('/')

free_percent = disk.free / disk.total * 100

if free_percent < DISK_THRESHOLD:
    print("Low Disk Space Alert")
else:
    print("Disk usage normal")
```

6.4.3 Experiment Procedure

A large file was created to reduce disk space:

```
fallocate -l 5G testfile
```

6.4.4 Observations

- Disk usage increased significantly
- Free space dropped below threshold
- Script detected condition correctly

6.4.5 Result

The disk monitoring system successfully identified low disk space conditions and generated alerts.

6.5 Memory Monitoring Experiment

6.5.1 Purpose

To monitor memory usage and detect overload conditions.

6.5.2 Code Explanation

```
import psutil
memory = psutil.virtual_memory()
if memory.percent > 80:
    print("High Memory Usage Alert")
else:
    print("Memory usage normal")
```

6.5.3 Observations

- Memory usage increased during heavy processing
- Script correctly detected high usage

6.5.4 Result

Memory monitoring worked accurately and efficiently.

6.6 Login Monitoring Experiment (Security Monitoring)

6.6.1 Purpose

To detect unauthorized login attempts and enhance system security.

6.6.2 Code Explanation

```
import subprocess

log = subprocess.getoutput("grep 'Failed password' /var/log/auth.log")

if log:
    print("Failed login attempts detected")
else:
    print("No failed login attempts")
```

6.6.3 Experiment Procedure

A failed login was simulated:

ssh wronguser@IP

6.6.4 Verification

sudo tail -n 20 /var/log/auth.log

6.6.5 Observations

- Failed login entry recorded in log
- Script detected failure
- Alert message generated

6.6.6 Result

The system successfully identified unauthorized login attempts.

6.7 Automation Experiment Using Cron

6.7.1 Purpose

To verify automatic execution of monitoring scripts.

6.7.2 Configuration

crontab -e

Entries:

*/5 * * * * cpu_monitor.py

*/5 * * * * disk_monitor.py

*/10 * * * * login_monitor.py

6.7.3 Observations

- Scripts executed automatically
- Logs updated periodically
- No manual execution required

6.7.4 Result

Automation was successful and ensured continuous monitoring.

6.8 Alert System Experiment

6.8.1 Purpose

To verify alert generation using Azure Monitor.

6.8.2 Setup

- Alert rule created in Azure
- Condition: CPU > 80%
- Action: Email notification

6.8.3 Observations

- CPU stress triggered alert
- Azure showed alert as “Fired”
- Email notification received

6.8.4 Result

Alert system worked correctly and provided real-time notifications.

6.9 Performance Analysis

Speed:

- Scripts executed quickly
- Minimal delay in detection

Accuracy:

- All conditions detected correctly

Resource Usage:

- Low CPU and memory usage

6.10 Summary of Experimentation

Experiment	Result
CPU Monitoring	Successful
Disk Monitoring	Successful
Memory Monitoring	Successful
Login Detection	Successful
Automation	Successful
Alert System	Successful

6.11 Conclusion of Experimentation

The experimentation phase proves that the Hybrid Cloud Monitoring System is:

- Accurate
- Efficient
- Reliable
- Scalable

The system successfully detects system anomalies and provides timely alerts, making it suitable for real-world cloud environments.

RESULTS

7.1 Overview of Results

The Hybrid Cloud Monitoring and Alerting System was successfully implemented and tested on a Microsoft Azure virtual machine. The results obtained from the experimentation phase demonstrate that the system is capable of monitoring system performance, detecting anomalies, and generating alerts in real time.

The results are categorized into:

- Performance Monitoring Results
- Security Monitoring Results
- Automation Results
- Cloud Alert Results
- Overall System Evaluation

7.2 Performance Monitoring Results

7.2.1 CPU Monitoring

The CPU monitoring module was tested by generating artificial load using multiple background processes.

Observations:

- CPU utilization increased to nearly 100%
- Monitoring script detected threshold breach
- Alert message was generated
- Azure Monitor triggered alert notification

Result:

The CPU monitoring system successfully identified high CPU usage and generated alerts without delay. This confirms that the system can handle real-time performance monitoring.

7.2.2 Disk Monitoring

Disk usage was tested by creating large files to reduce available storage.

Observations:

- Disk space reduced significantly
- Free space dropped below threshold

- Monitoring script detected condition
- Warning message generated

Result:

The disk monitoring system accurately detected low disk space conditions and generated appropriate alerts, ensuring system stability.

7.2.3 Memory Monitoring

Memory usage was observed during high workload conditions.

Observations:

- Memory usage increased during execution
- Script detected high usage levels
- Alert message generated

Result:

The memory monitoring module successfully tracked RAM utilization and detected overload conditions effectively.

7.3 Security Monitoring Results

7.3.1 Login Failure Detection

The system was tested by simulating unauthorized login attempts.

Observations:

- Failed login attempts were recorded in /var/log/auth.log
- Monitoring script detected suspicious activity
- Alert message generated

Result:

The login monitoring module successfully detected unauthorized access attempts, improving system security and reliability.

7.4 Automation Results

The automation of monitoring scripts was tested using cron jobs.

Observations:

- Scripts executed automatically at scheduled intervals

- Monitoring was continuous without manual intervention
- Log files were updated periodically

Result:

The automation system worked efficiently, ensuring continuous monitoring and reducing manual effort.

7.5 Cloud Alert Results (Azure Monitor)

Azure Monitor was configured to generate alerts when CPU usage exceeded the threshold.

Observations:

- High CPU usage triggered Azure alert rule
- Alert status changed to “Fired” in Azure Portal
- Email notification was successfully received

Result:

The integration with Azure Monitor was successful and provided real-time alerting capabilities.

7.6 Log Analysis Results

System logs generated by monitoring scripts were analysed.

Observations:

- Logs contained accurate system status information
- Alerts were recorded correctly
- No missing or incorrect entries were found

Result:

The logging mechanism was reliable and provided useful data for system analysis and debugging.

7.7 Performance Evaluation

Speed

- Monitoring scripts executed quickly
- Minimal delay in detection

Accuracy

- All threshold conditions were correctly identified
- No false alerts were observed

Resource Usage

- Scripts consumed minimal CPU and memory
- No impact on system performance

7.8 Comparative Analysis

Feature	Azure Monitor	Python Scripts	Hybrid System
CPU Monitoring	Yes	Yes	Yes
Disk Monitoring	Limited	Yes	Yes
Login Monitoring	No	Yes	Yes
Custom Alerts	Limited	Yes	Yes
Automation	Yes	Yes	Yes

Conclusion:

The hybrid approach provides better monitoring capabilities compared to using Azure Monitor alone.

7.9 Key Outcomes

The system successfully achieved:

- Real-time monitoring of system performance
- Detection of abnormal conditions
- Identification of security threats
- Automated execution of monitoring tasks
- Integration with cloud alert systems

7.10 Overall Result Summary

Module	Status
CPU Monitoring	Successful
Disk Monitoring	Successful
Memory Monitoring	Successful
Login Monitoring	Successful
Automation	Successful

Module	Status
Alert System	Successful

7.11 Final Result Conclusion

The results clearly demonstrate that the Hybrid Cloud Monitoring System is:

- Efficient in monitoring system performance
- Effective in detecting security threats
- Reliable in generating alerts
- Scalable for cloud environments

The system provides a comprehensive monitoring solution by combining cloud-based tools with custom scripts, making it suitable for real-world applications.

Conclusion

8.1 Overview

The Hybrid Cloud Monitoring and Alerting System successfully demonstrates how cloud-native monitoring tools can be combined with custom scripting techniques to achieve a comprehensive and efficient monitoring solution. The project was implemented using Microsoft Azure and Python, providing both infrastructure-level and operating system-level monitoring capabilities.

The system was able to monitor critical parameters such as CPU usage, disk utilization, memory usage, and failed login attempts. By integrating Azure Monitor with Python-based scripts, the system overcomes the limitations of standalone cloud monitoring tools and provides deeper visibility into system operations.

8.2 Achievement of Objectives

The project successfully achieved all its intended objectives:

- Continuous monitoring of system performance metrics
- Detection of abnormal conditions such as high CPU and low disk space
- Identification of unauthorized login attempts
- Automation of monitoring tasks using cron jobs
- Real-time alert generation using Azure Monitor

These outcomes confirm that the system functions effectively and meets the requirements of a real-world monitoring solution.

8.3 Key Contributions of the Project

The major contributions of this project include:

1. Hybrid Monitoring Approach

The project introduces a hybrid architecture that combines cloud monitoring with custom scripting. This approach enhances monitoring capabilities and provides better control over system behaviour.

2. Real-Time Alerting Mechanism

The system successfully demonstrates real-time alert generation through Azure Monitor. This ensures that administrators are immediately notified of critical system issues.

3. Security Enhancement

The implementation of login monitoring improves system security by detecting failed login attempts and potential unauthorized access.

4. Automation

The use of cron jobs ensures continuous and automatic execution of monitoring scripts, reducing the need for manual intervention.

8.4 Performance Evaluation Summary

The system was evaluated based on several factors:

Efficiency

The monitoring scripts executed quickly and consumed minimal system resources.

Accuracy

The system correctly detected all threshold violations and generated accurate alerts.

Reliability

The system operated continuously without failure, ensuring consistent monitoring.

Scalability

The system can be extended to monitor multiple virtual machines, making it suitable for enterprise environments.

8.5 Advantages of the System

- Provides real-time system monitoring
- Enhances system security
- Automates monitoring tasks
- Offers flexible and customizable monitoring
- Integrates with cloud alert systems
- Low cost and easy to implement

8.6 Limitations

Despite its advantages, the system has some limitations:

- Requires manual configuration of scripts
- Limited visualization compared to advanced monitoring tools
- Depends on proper configuration of cron jobs
- Basic alerting without advanced analytics

8.7 Future Enhancements

The system can be further improved by adding:

- Integration with dashboards for visualization
- AI/ML-based anomaly detection
- Centralized monitoring for multiple systems
- Integration with communication tools like Teams or Slack
- Real-time streaming analytics

8.8 Final Conclusion

In conclusion, the Hybrid Cloud Monitoring and Alerting System provides an effective and practical solution for monitoring cloud-based virtual machines. By combining Azure Monitor with custom Python scripts, the system achieves comprehensive monitoring, improved security, and real-time alerting.

The project reflects real-world industry practices and demonstrates how cloud technologies can be enhanced with custom solutions to build scalable and efficient monitoring systems. It serves as a strong foundation for future development in cloud monitoring, automation, and intelligent system management.

References

1. https://www.researchgate.net/publication/257354329_Cloud_Network_Security_Monitoring_and_Response_System
2. https://gjeta.com/sites/default/files/GJETA-2025-0059.pdf
3. https://ijritcc.org/index.php/ijritcc/article/view/11386
4. https://www.sciencedirect.com/science/article/pii/S0950584920301452
5. https://www.scitepress.org/Papers/2024/126908/126908.pdf
6. https://arxiv.org/abs/1803.05233